网络协议分析实践-DNS解析器

计算机学院（国家示范性软件学院）

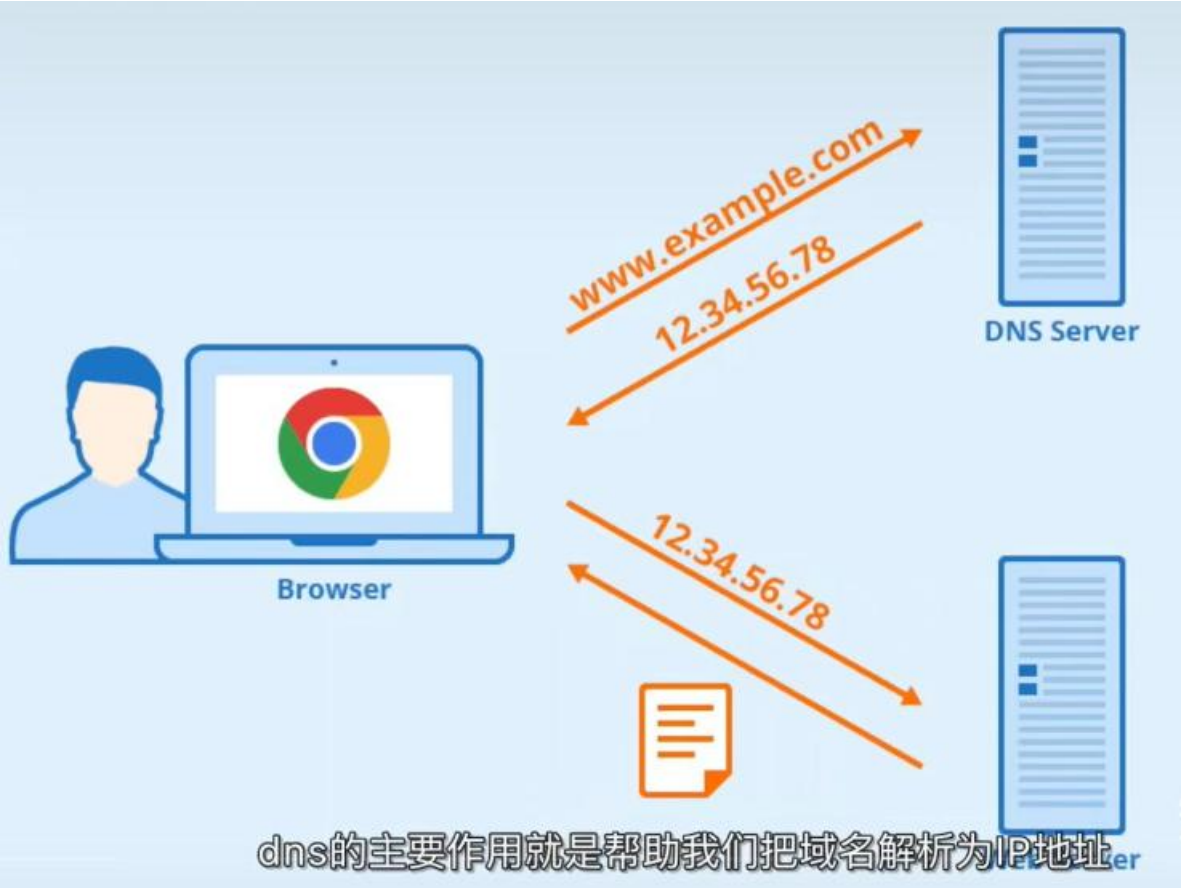
目录

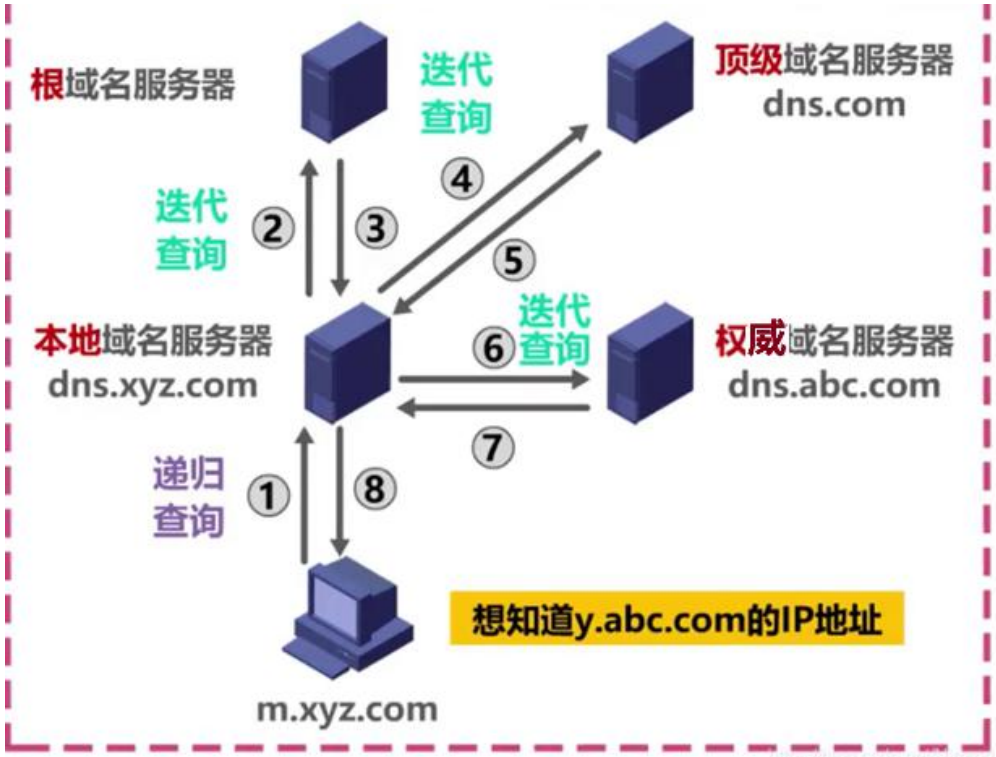
01 • DNS协议基础

02 • 利用libpcap库进行数据包收发

03 • 数据包的构造与解析

DNS协议基础


www.example.com
12.34.56.78
DNS Server
Browser
12.34.56.78
dns的主要作用就是帮助我们把域名解析为IP地址


根域名服务器
迭代
查询
顶级域名服务器
dns.com
迭代
查询
②
③
④
⑤
本地域名服务器
dns.xyz.com
⑥
迭代
查询
权威域名服务器
dns.abc.com
⑦
递归
查询
①
⑧
m.xyz.com
想知道y.abc.com的IP地址

DNS协议基础

DNS记录可以理解成一个键值对：

- 键：域名；
- 值：与域名关联的值；

- A： 主机 IP 地址；
- AAAA： 主机 IPv6 地址；
- ALIAS： 自动解析的别名（*alias*）；
- CNAME： 别名的权威名称（*canonical name*
- MX： 邮件交换服务器（*Mail eXchange*）
- NS： 域名服务器（*name server*）；
- TXT： 描述文本；
- **SRV: 定义特定服务的主机和端口**

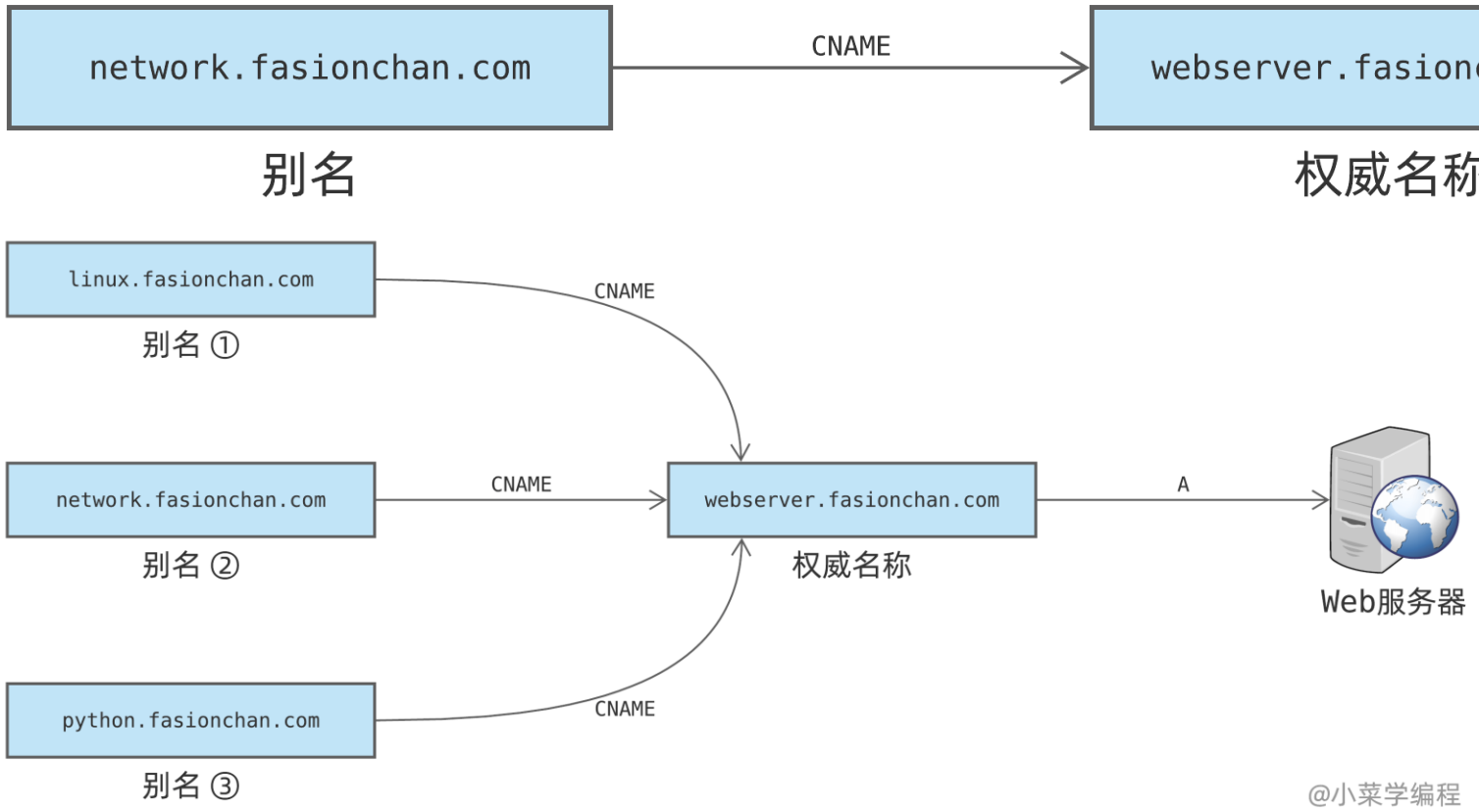
```
root@netbox [ ~ ]  ➜ dig test.fasionchan.com

; <<>> DiG 9.16.1-Ubuntu <<>> test.fasionchan.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 49579
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;test.fasionchan.com.       IN  A

;; ANSWER SECTION:
test.fasionchan.com.    752 IN  A   10.0.0.1

;; Query time: 71 msec
;; SERVER: 192.168.65.1#53(192.168.65.1)
;; WHEN: Mon Apr 26 17:22:16 CST 2021
;; MSG SIZE  rcvd: 53
```

DNS协议基础

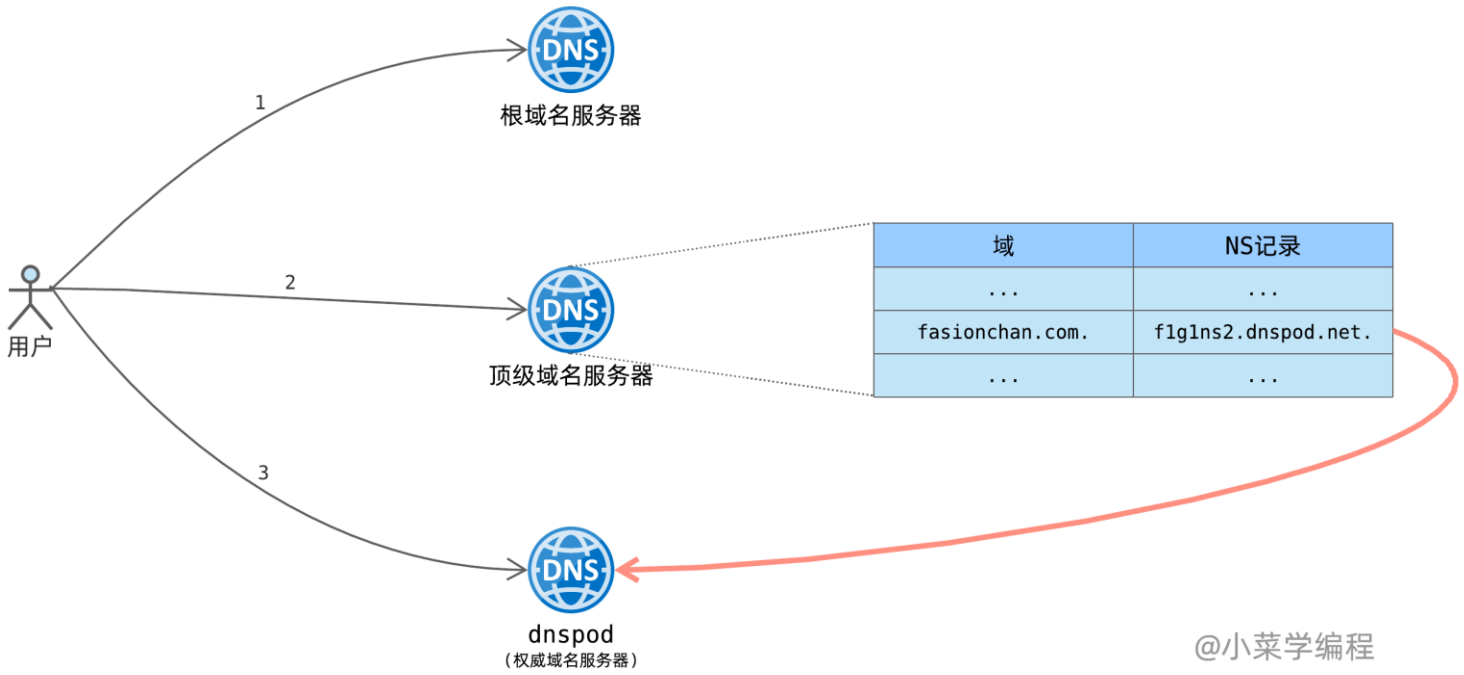
```
root@netbox [ ~ ]  ➜ dig t-aaaa.fasionchan.com AAAA

; <<>> DiG 9.16.1-Ubuntu <<>> t-aaaa.fasionchan.com AAAA
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 5088
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;t-aaaa.fasionchan.com.     IN  AAAA

;; ANSWER SECTION:
t-aaaa.fasionchan.com.  752 IN  AAAA    ::1

;; Query time: 50 msec
;; SERVER: 192.168.65.1#53(192.168.65.1)
;; WHEN: Mon Apr 26 17:37:10 CST 2021
;; MSG SIZE  rcvd: 67
```

DNS协议基础

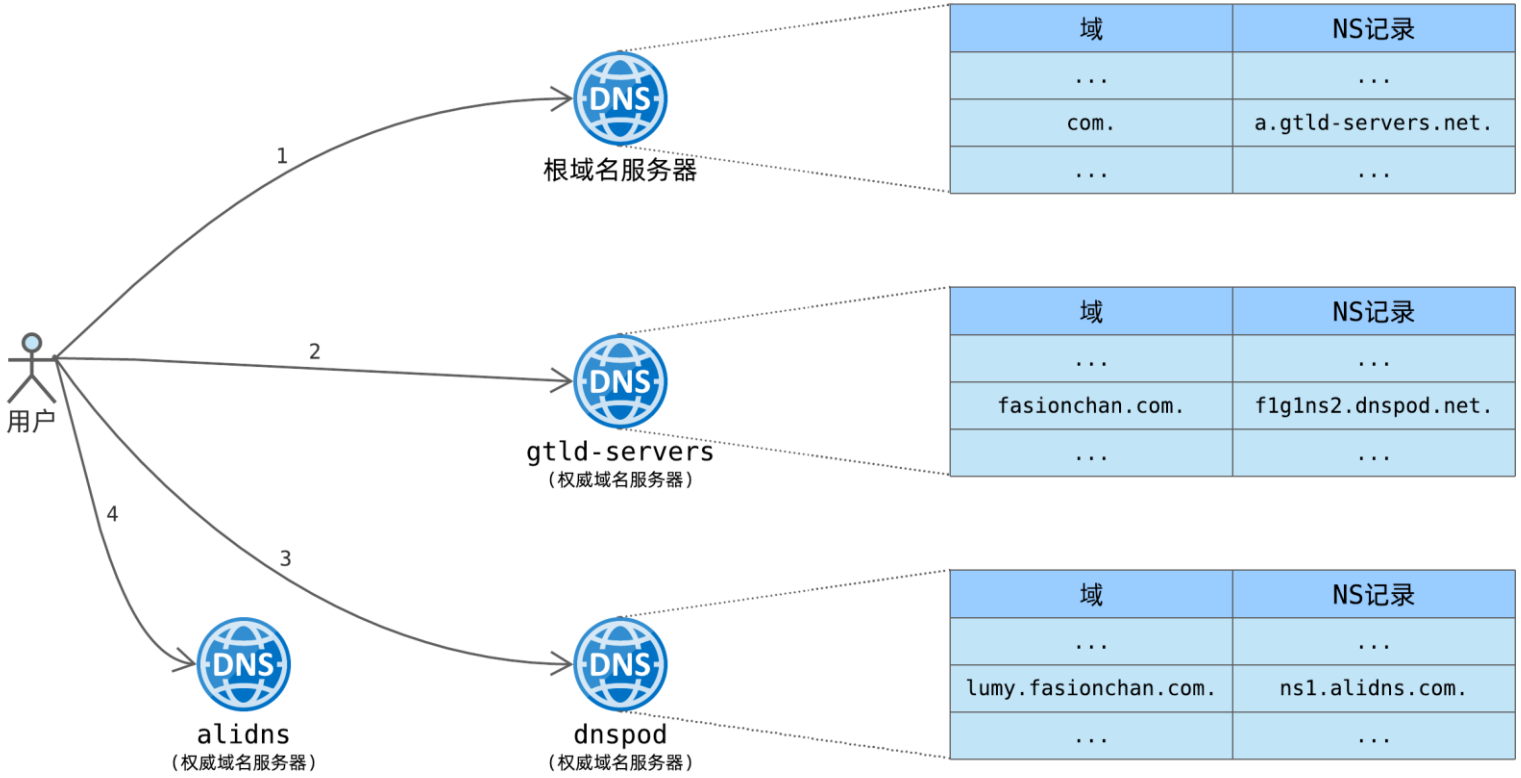
域名可以取别名，以 webserver.fasionchan.com 为例，它可以取一个别名，比如：network.fasionchan.com 。如此一来，我们称前者是后者的 权威名称 ， CNAME 记录则保存权威名称。



DNS协议基础


1
2
3
用户
根域名服务器
顶级域名服务器
dnspod
（权威域名服务器）
域
NS记录
...
...
fasionchan.com.
f1g1ns2.dnspod.net.
...
...
@小菜学编程

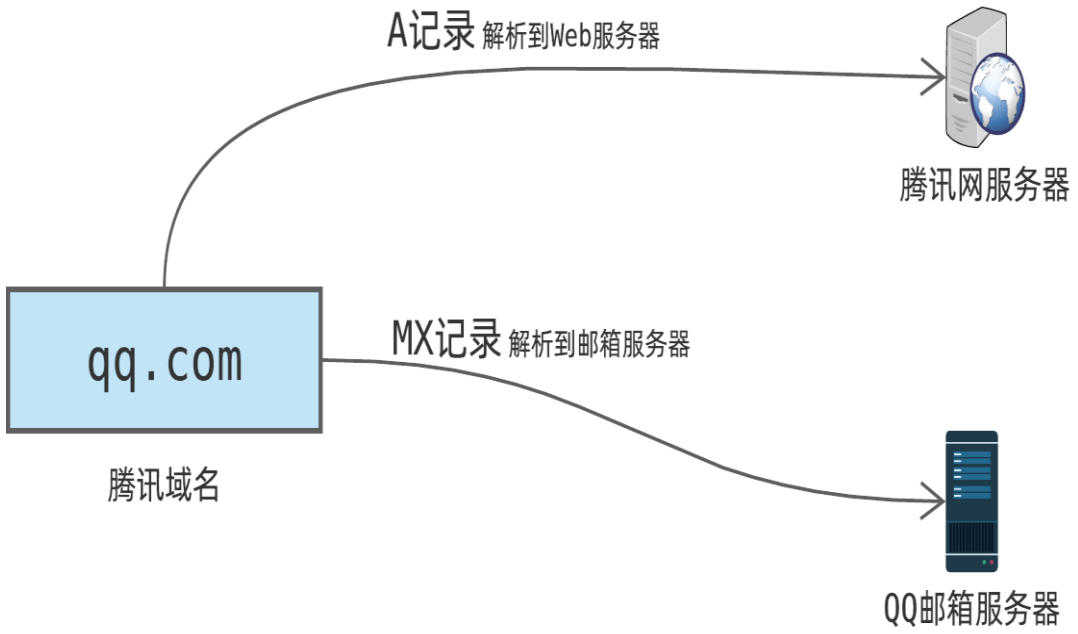
DNS协议基础



@小菜学编程

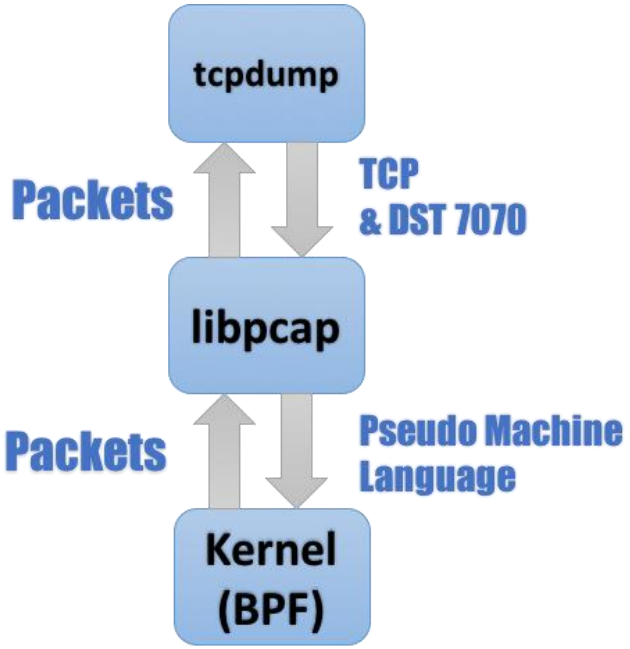
DNS协议基础

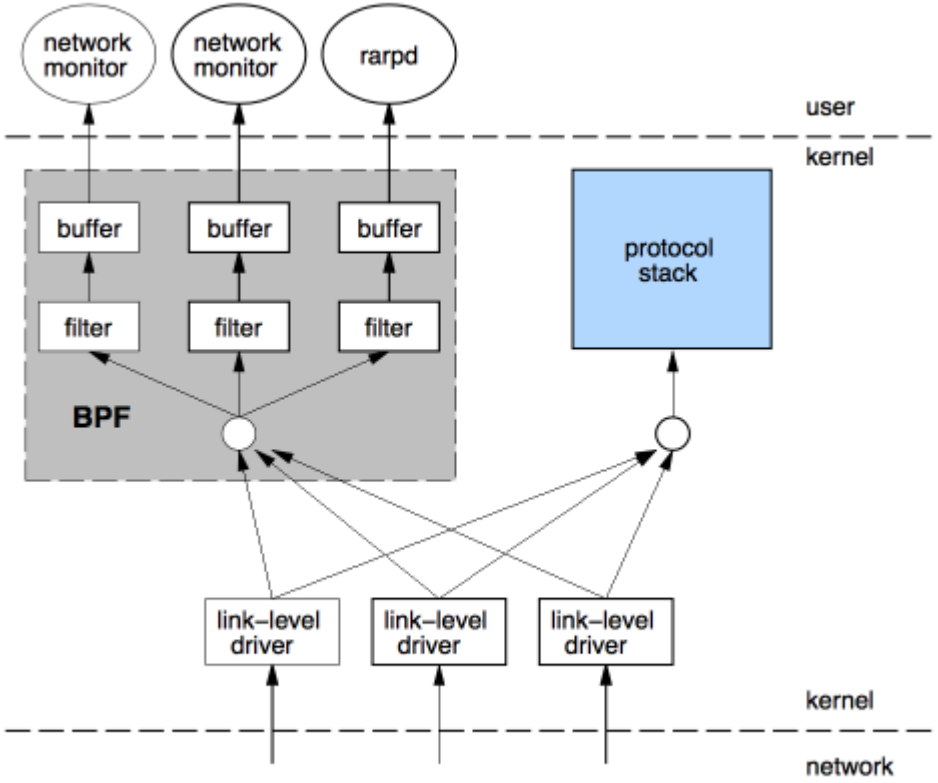
```
root@netbox [ ~ ]  → dig qq.com MX

; <<>> DiG 9.16.1-Ubuntu <<>> qq.com MX
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 49462
;; flags: qr rd ra; QUERY: 1, ANSWER: 3, AUTHORITY: 0, ADDITIONAL: 4

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4000
;; QUESTION SECTION:
;qq.com.                IN   MX

;; ANSWER SECTION:
qq.com.         1435    IN   MX   20 mx2.qq.com.
qq.com.         1435    IN   MX   10 mx3.qq.com.
qq.com.         1435    IN   MX   30 mx1.qq.com.

;; ADDITIONAL SECTION:
mx3.qq.com.     463 IN  A    113.96.208.206
mx3.qq.com.     3   IN  AAAA     240e:ff:f101:10::127
mx1.qq.com.     2409    IN   A    14.215.140.20

;; Query time: 18 msec
;; SERVER: 10.2.66.66#53(10.2.66.66)
;; WHEN: Wed Apr 28 18:40:03 CST 2021
;; MSG SIZE  rcvd: 155
```



libpcap库

libpcap是一个广泛使用的网络抓包库，提供了一种在各种操作系统上捕获和分析网络数据包的统一接口。它以平台无关的方式实现，因此广泛用于网络监控、协议分析、流量管理和网络安全等领域。libpcap不仅可以捕获传输中的数据包，还可以对这些数据包进行过滤和分析。





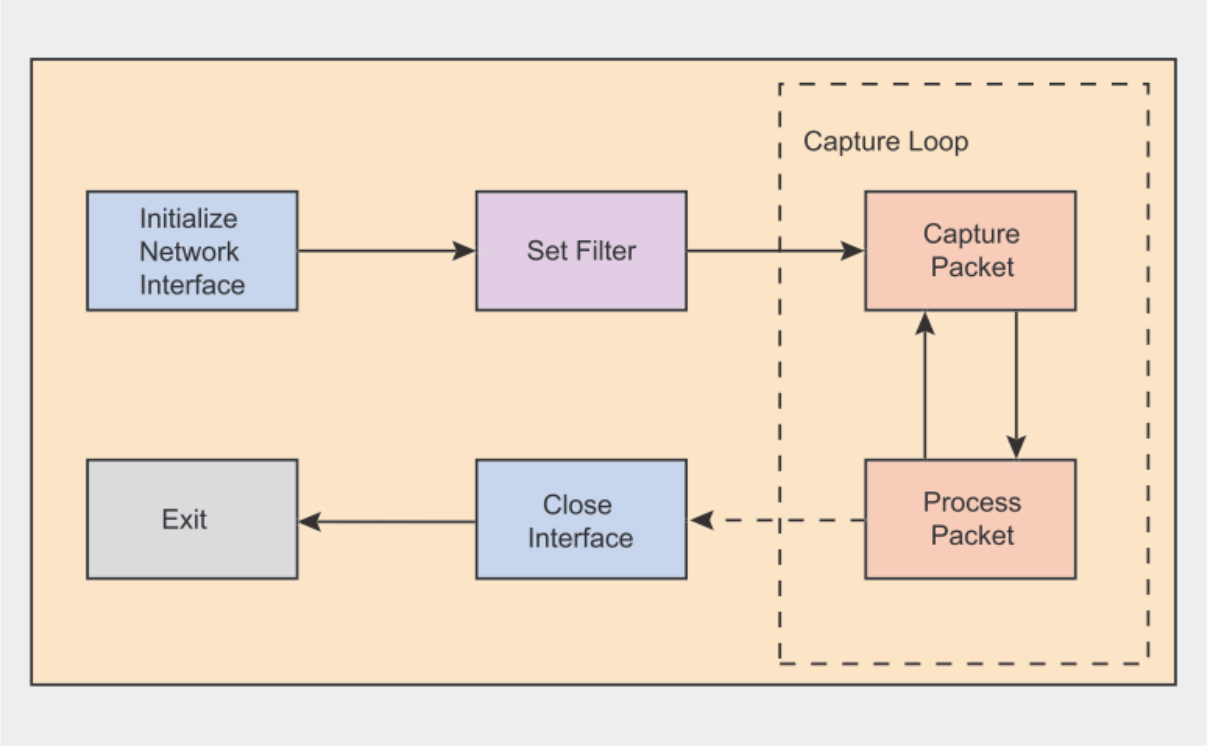
libpcap库

1）**确定将要嗅探的接口**，在linux下是类似eth0的东西。在BSD下是类似xll的东西。可以在一个字符串中声明设备，也可以让pcap提供备选接口（我们想要嗅探的接口）的名字。

（2）**初始化pcap**，此时才真正告诉pcap我们要嗅探的具体接口，只要我们愿意，我们可以嗅探多个接口。但是如何区分多个接口呢，使用文件句柄。就像读写文件时使用文件句柄一样。我们必须给嗅探任务命名，以至于区分不同的嗅探任务。

（3）**指定过滤规则**，当我们只想嗅探特殊的流量时（例如，仅仅嗅探TCP/IP包、仅仅嗅探经过端口80的包，等等）我们必须设定一个规则集，“编译”并应用它。这是一个三相的并且紧密联系的过程，规则集存储与字符串中，在“编译”之后会转换成pcap可以读取的格式。“编译过程”实际上是调用自定义的函数完成的，不涉及外部的函数。然后我们可以告诉pcap在我们想要过滤的任何任务上实施。

（4）**抓包**，最后，告诉pcap进入主要的执行循环中，在此阶段，在接收到任何我们想要的包之前pcap将一直循环等待。在每次抓取到一个新的数据包时，它将调用另一个自定义的函数，我们可以在这个函数中肆意妄为，例如，解析数据包并显示数据内容、保存到文件或者什么都不做等等。



libpcap库

pcap_lookupdev()函数用于查找网络设备，返回可被pcap_open_live()函数调用的网络设备名指针。

pcap_open_live()函数用于打开网络设备，并且返回用于捕获网络数据包的数据包捕获描述字。对于此网络设备的操作都要基于此网络设备描述字。

pcap_lookupnet()函数获得指定网络设备的网络号和掩码。

pcap_compile()函数用于将用户制定的过滤策略编译到过滤程序中。

pcap_setfilter()函数用于设置过滤器。

pcap_loop()函数pcap_dispatch()函数用于捕获数据包，捕获后还可以进行处理，此外pcap_next()和pcap_next_ex()两个函数也可以用来捕获数据包。

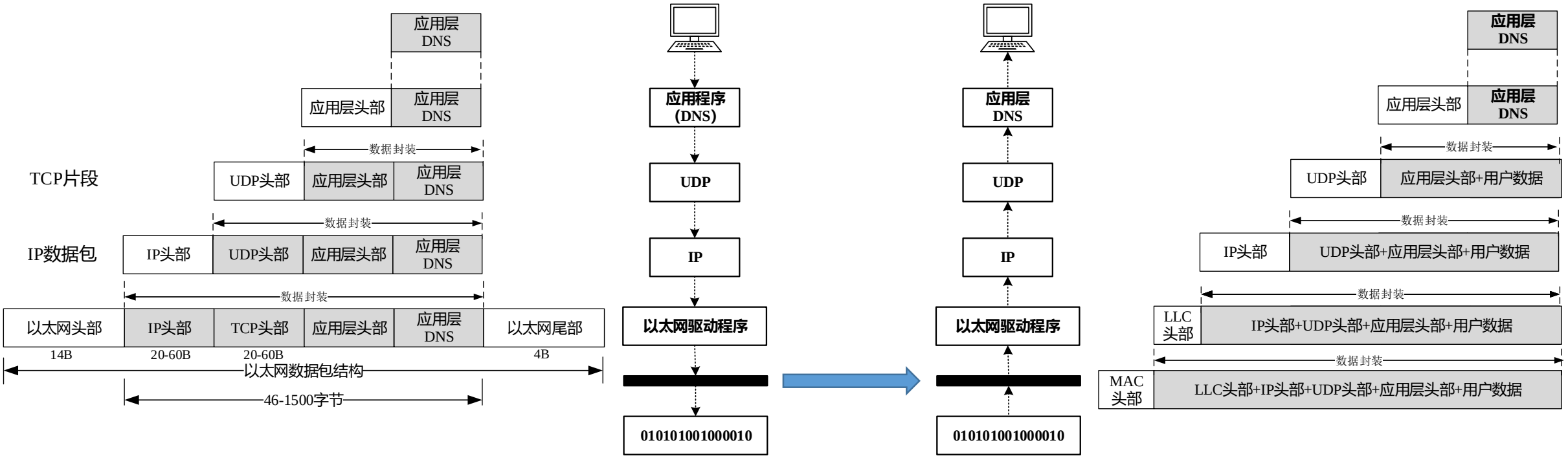
pcap_close()函数用于关闭网络设备，释放资源。

构造数据包库（pcap）：

Linux： libpcap

Windows：npcap

网络协议基础知识与实践—DNS请求工具



DNS记录格式：

- A记录
- AAAA记录
- C记录
- NS记录

message header

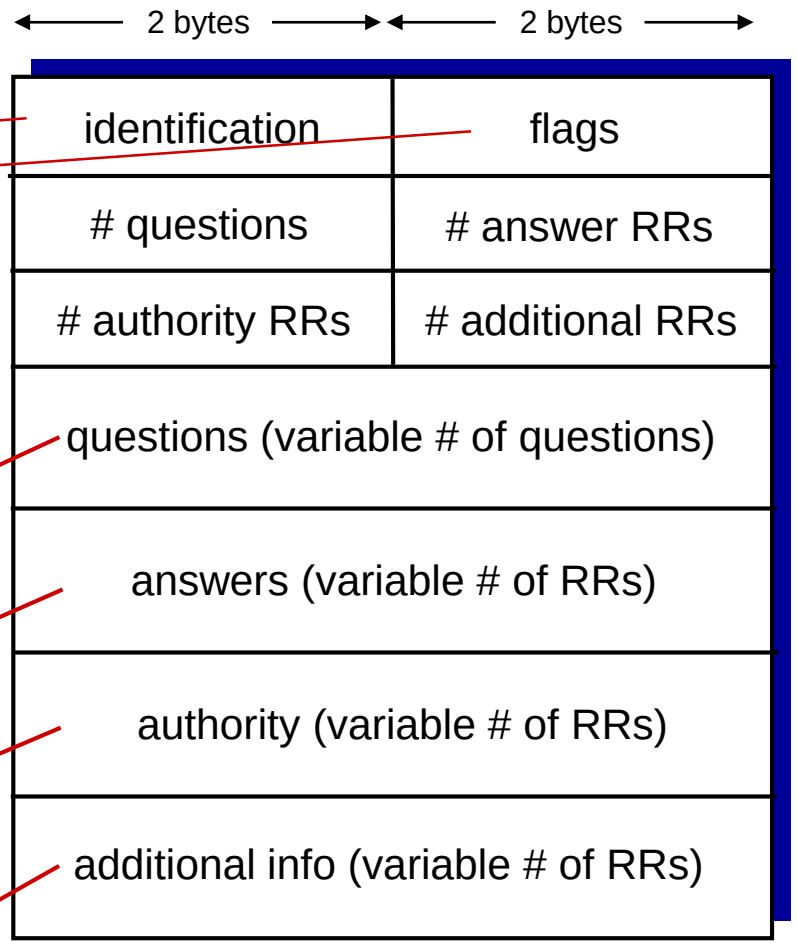
- identification: 16 bit # for query, reply to query uses same #
- flags:
  - query or reply
  - recursion desired
  - recursion available
  - reply is authoritative

name, type fields for a query

RRs in response to query

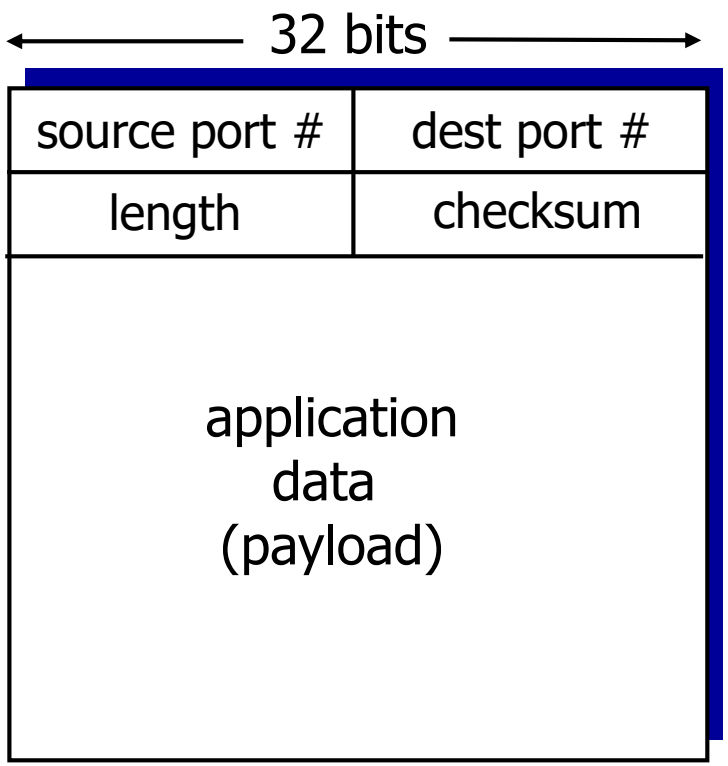
records for authoritative servers

additional "helpful" info that may be used



2 UDP协议

协议格式

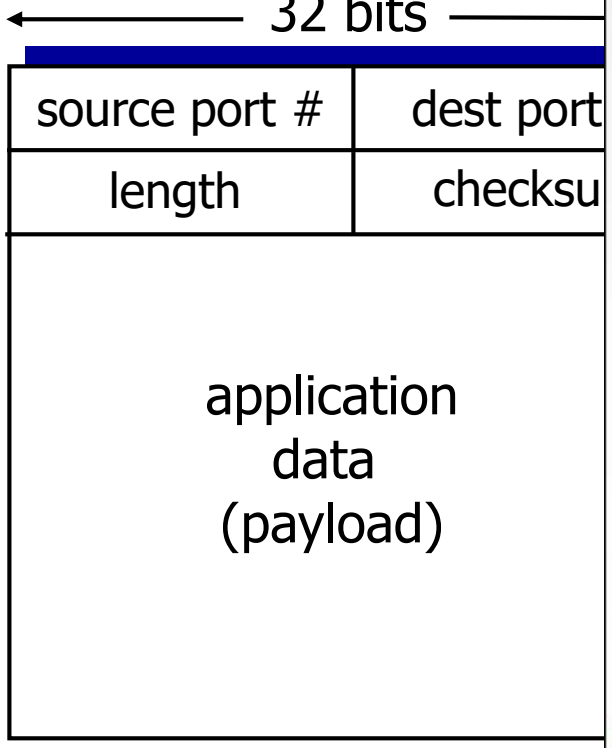


UDP segment format

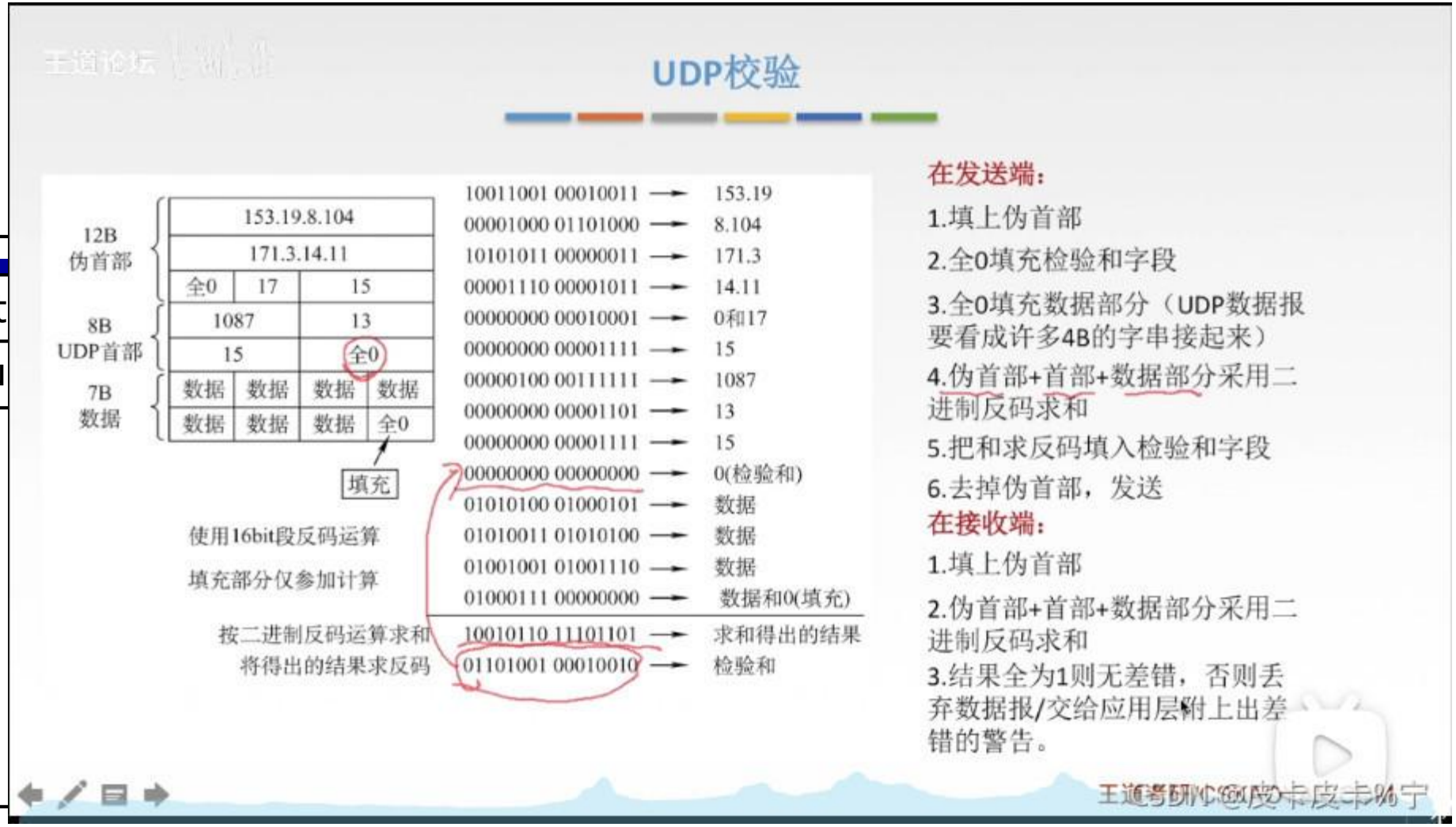
```
/* UDP header as specified by RFC 768, August 1980. */
#ifdef __FAVOR_BSD

struct udphdr
{
  u_int16_t uh_sport;          /* source port */
  u_int16_t uh_dport;          /* destination port */
  u_int16_t uh_ulen;           /* udp length */
  u_int16_t uh_sum;            /* udp checksum */
};

#else

struct udphdr
{
  u_int16_t source;
  u_int16_t dest;
  u_int16_t len;
  u_int16_t check;
};
#endif
```

2 UDP协议

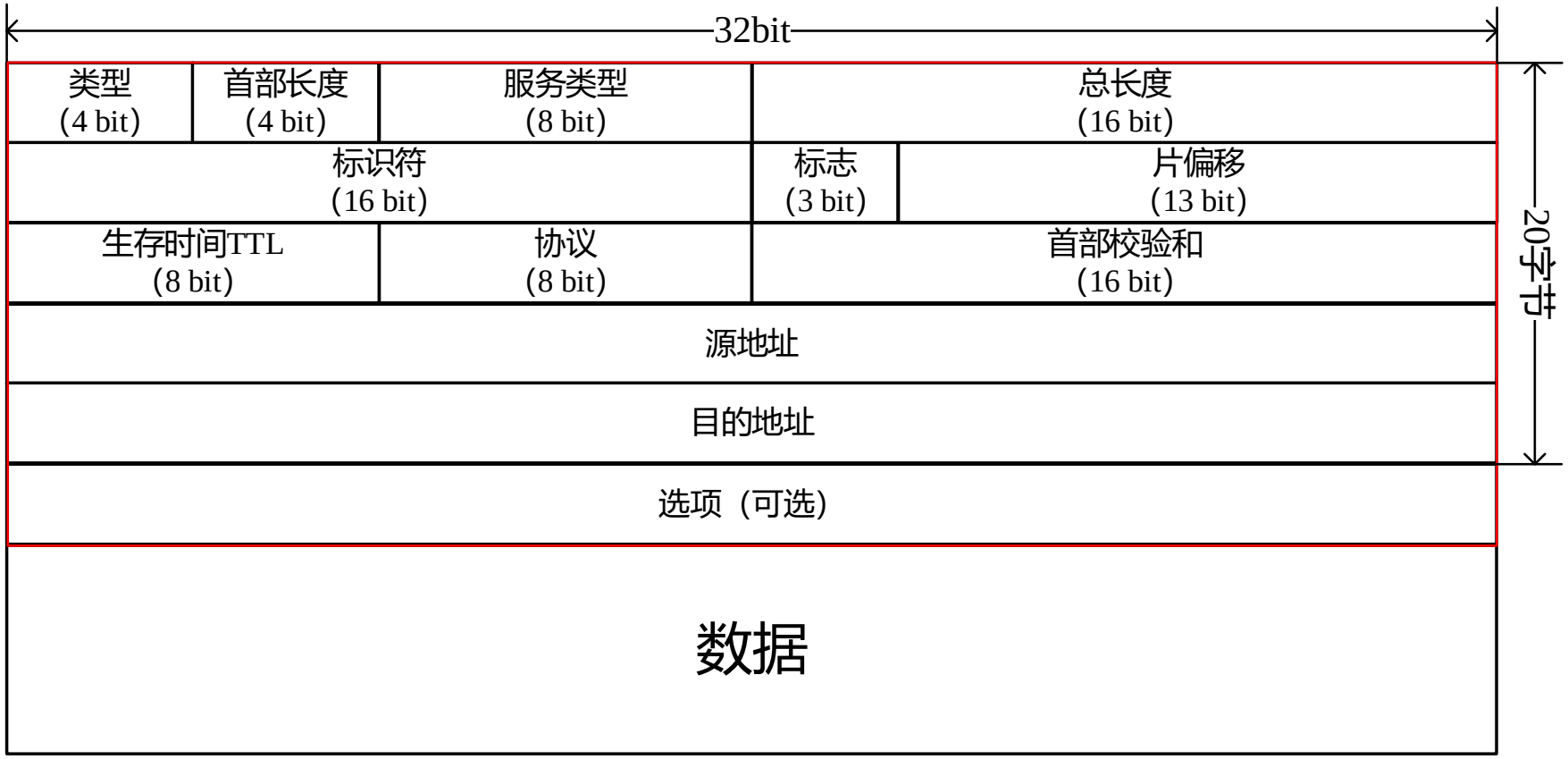
协议格式



UDP segment format



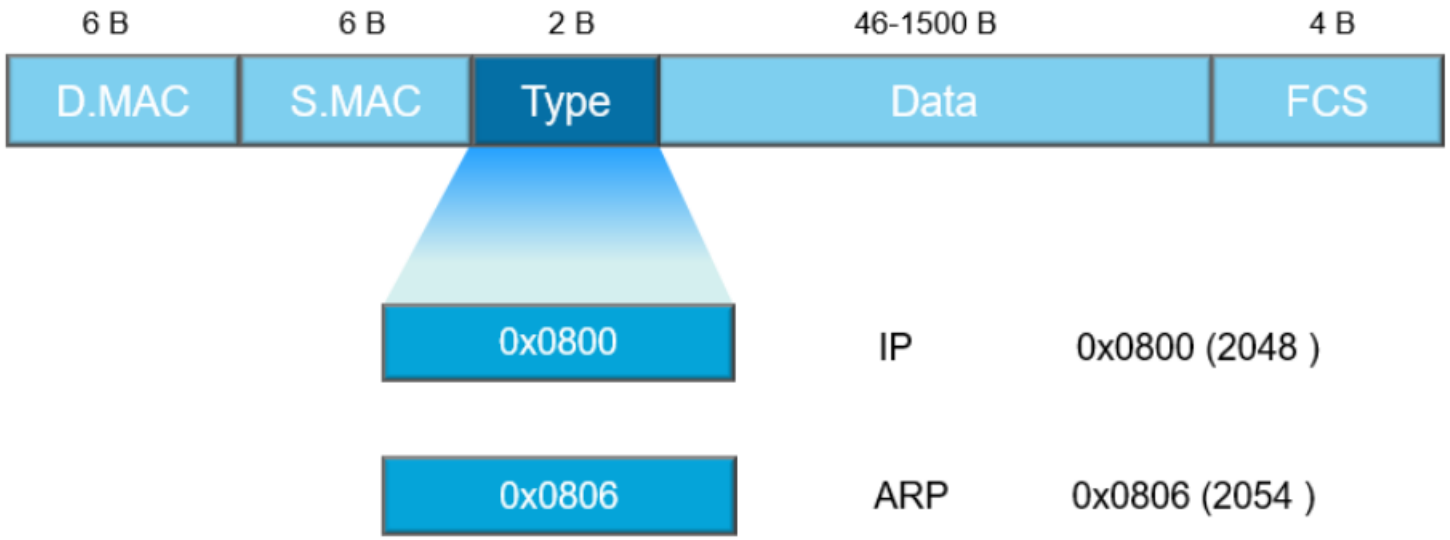
2 IP协议


32bit
类型
(4 bit)
首部长度
(4 bit)
服务类型
(8 bit)
总长度
(16 bit)
标识符
(16 bit)
标志
(3 bit)
片偏移
(13 bit)
生存时间TTL
(8 bit)
协议
(8 bit)
首部校验和
(16 bit)
源地址
目的地址
选项（可选）
数据
20字节

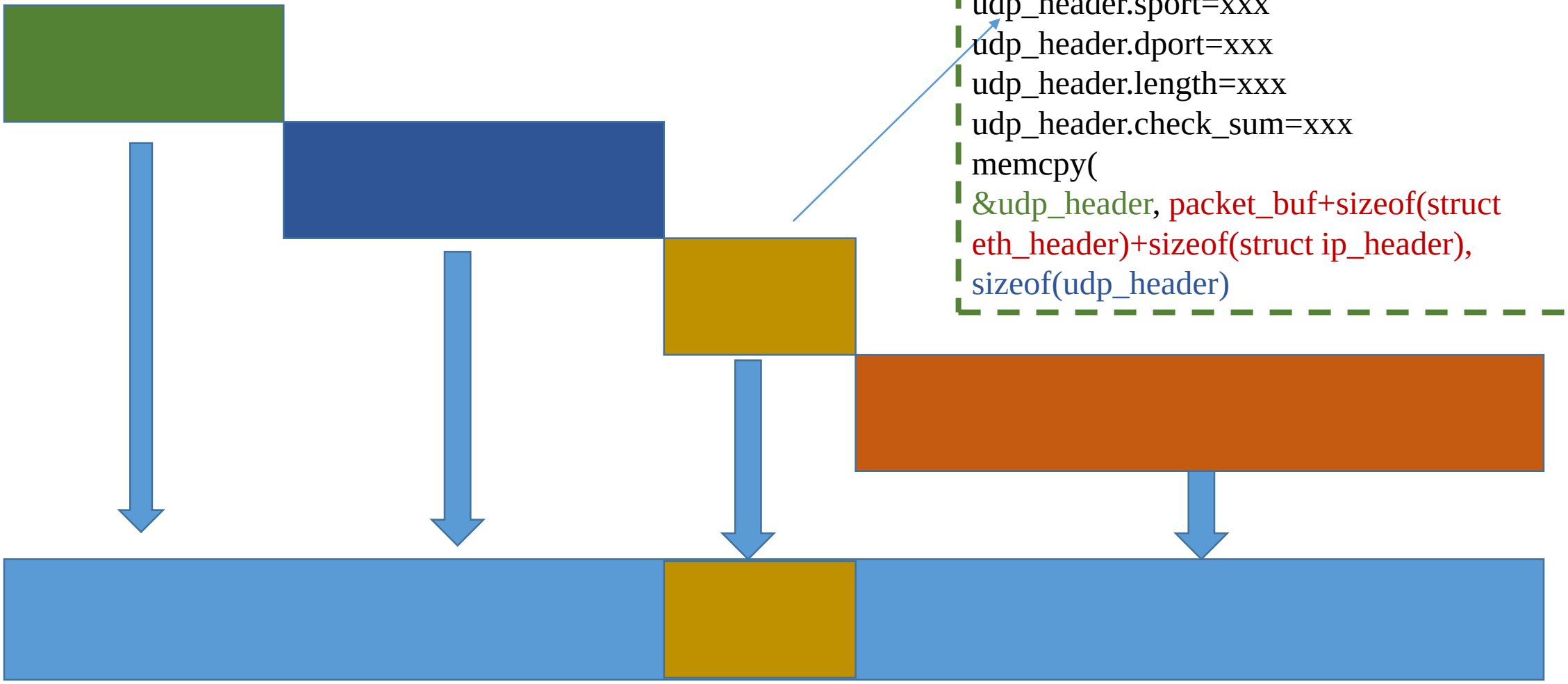
网络协议开发基础知识与实践—校验和

```
unsigned short cal_chksum(unsigned short *addr, int len)
{
    int nleft = len;
    int sum = 0;
    unsigned short *w=addr;
    unsigned short answer = 0;
    while(nleft > 1){    // 16bit为单位累加运算
        sum += *w++;
        nleft -= 2;
    }
    if(nleft == 1){  //若addr奇数个字节,会剩下最后一字节.
       sum + =*(unsigned char *)w;
    }
    sum = (sum>>16) + (sum&0xffff);
    sum += (sum>>16);
    answer = ~sum;
    return answer;
}
```

网络协议开发基础知识与实践—以太帧



libpcap库



```
Struct UDP   udp_header;
udp_header.sport=xxx
udp_header.dport=xxx
udp_header.length=xxx
udp_header.check_sum=xxx
memcpy(
&udp_header, packet_buf+sizeof(struct
eth_header)+sizeof(struct ip_header),
sizeof(udp_header)
```

libpcap库

p_eth_header=(Struct ETH *)(packet_buf)

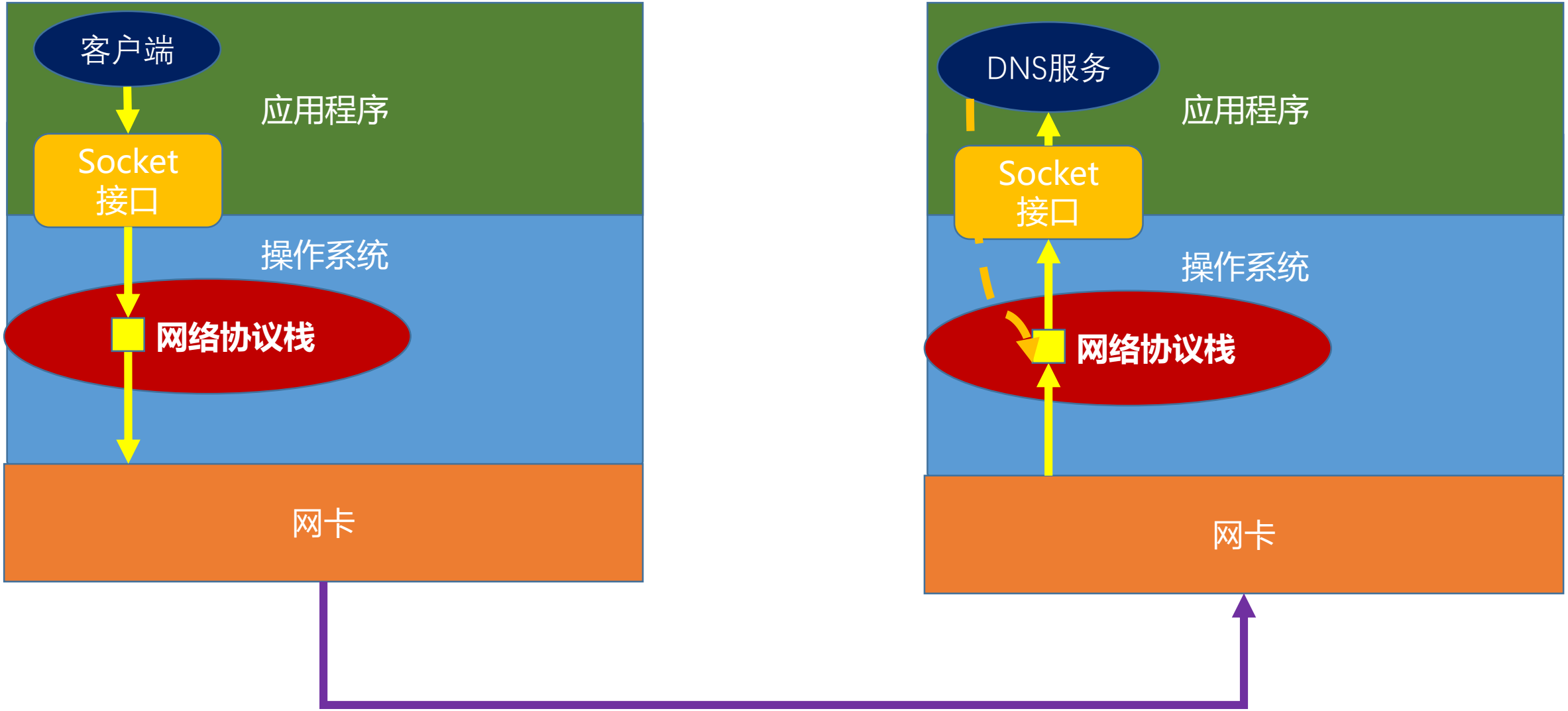
p_ip_header=(Struct IP *)(packet_buf+
sizeof(Struct ETH))

p_udp_header=(Struct UDP *)(packet_buf+
sizeof(Struct ETH)+
sizeof(struct IP))

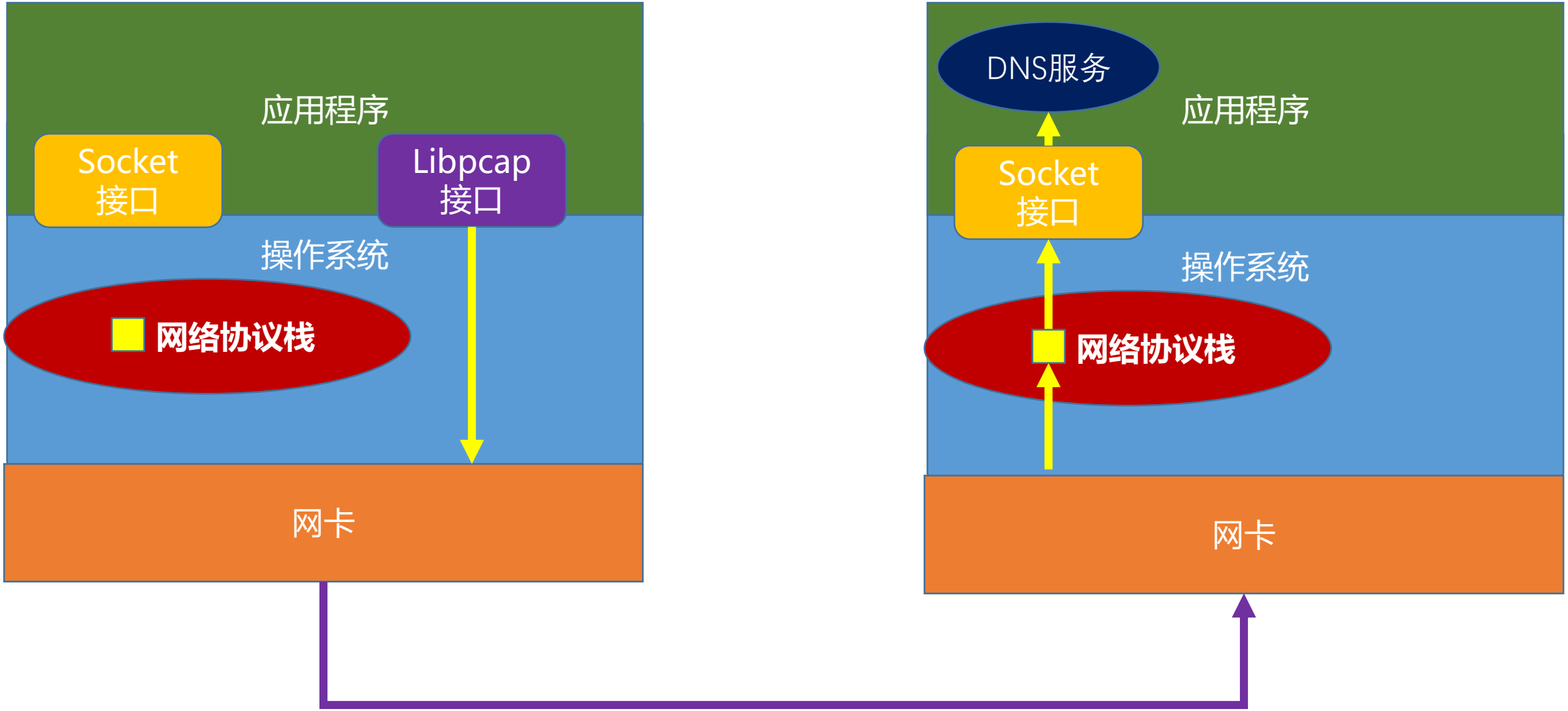
p_udp_header->sport=xxx
p_udp_header->dport=xxx
p_udp_header->length=xxx
p_udp_header->check_sum=xxx

p_dns_request=(Struct DNS_REQUEST *)(packet_buf+
sizeof(Struct ETH)+
sizeof(struct IP)+
sizeof(struct UDP))

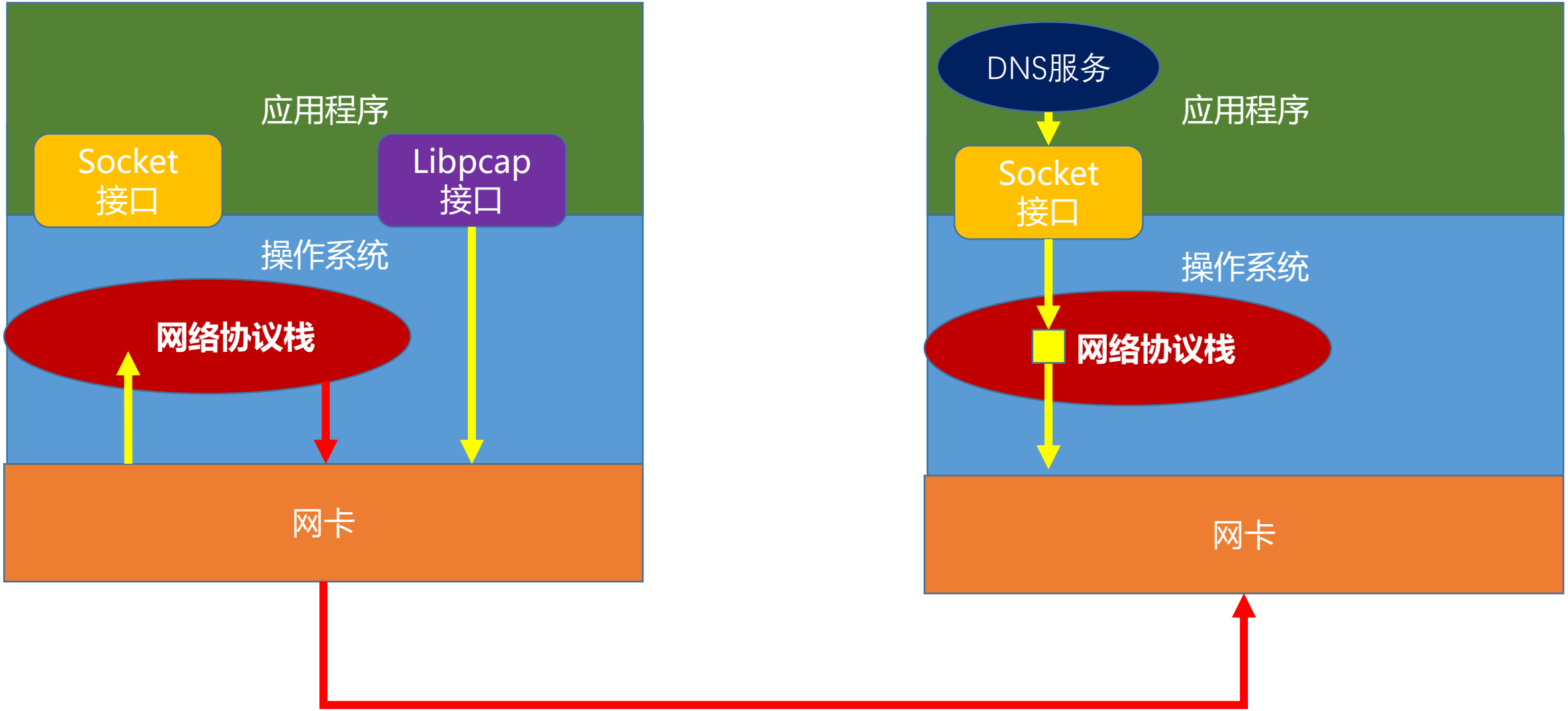
libpcap库


客户端
应用程序
Socket
接口
操作系统
网络协议栈
网卡
DNS服务
应用程序
Socket
接口
操作系统
网络协议栈
网卡

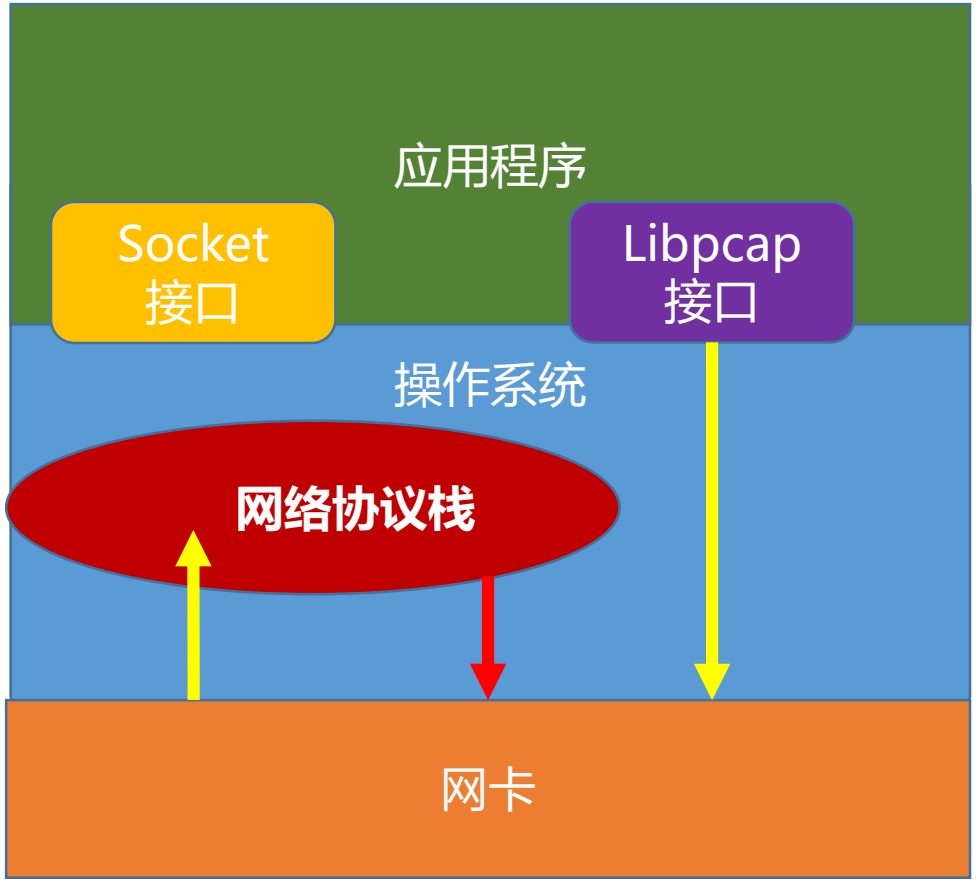
libpcap库


应用程序
Socket
接口
Libpcap
接口
操作系统
网络协议栈
网卡
DNS服务
应用程序
Socket
接口
操作系统
网络协议栈
网卡

libpcap库


应用程序
Socket
接口
Libpcap
接口
操作系统
网络协议栈
网卡
DNS服务
应用程序
Socket
接口
操作系统
网络协议栈
网卡

libpcap库



UDP：port unreachable

UDP的规则之一是，如果收到一份UDP数据报而目的端口与某个正在使用的进程不相符，那么UDP返回一个ICMP不可达报文。

TCP：RST

如果一个端口没有正在监听的服务，但TCP收到了一个根本不存在的连接上的报文，TCP处理程序会在自己认为的异常时刻发送RST包来关闭连接。

libpcap库

其他注意事项：

1 DNS的**压缩与指针机制**

2 发送和接收的独立运行（双线程）

3 自动获取网络接口信息，自动获取接口的IP地址，mac地址，下一跳mac地址（网关mac地址）

4 UDP校验和的正确计算

实践1

题目： 实现一个自定义的数据包解析工具

1 程序名称：my_nslookup

2 支持如下参数：

必选参数： DNS服务器IP地址，域名

可选参数： 本机mac地址，网关mac地址，本机IP地址，UDP源端口

3 程序能够向指定的DNS服务器发送域名请求，并解析服务的应答。

3.1 需根据协议的格式自行构造请求数据包的：应用层（DNS协议），传输层（UDP），网络层（IP），链路层（以太网），对发送数据包各层的主要字段进行打印。

3.2 使用数据包发送API将数据包发送到网络上，并捕获响应包

3.3 对响应包各层的主要字段进行打印。

3.4 将请求数据包和响应数据包保存成：pcap文件，并使用wireshark进行验证。

第一次实践评分标准

评分项	比例	基本要求	要点
实验报告	50%	团队独立完成并现场答辩。 在此基础上综合考虑用户友好度与程序优化程度，是否有创新思路或解决方案。	1 封装数据包的过程是否合理高效，层次清晰； 2 在实验过程中遇到的问题以及解决思路；
现场演示	50%	程序完成基本功能目标且能够正常稳定运行。 团队独立完成，源码格式规范且无明显Bug。	1 能够以友好的格式打印出链路层、网络层、传输层信息 2 针对UDP协议，打印DNS请求和DNS应答的全部信息。例如：请求域名，NS记录，C记录，A记录，AAAA记录；

源代码、报告文档、PPT发送至邮箱：40638183@qq.com
命名规范：
2026春网络协议分析实践实验一-第x组-源代码/PPT/实验报告

注：每次实验成绩按100分，占最终总成绩的50%